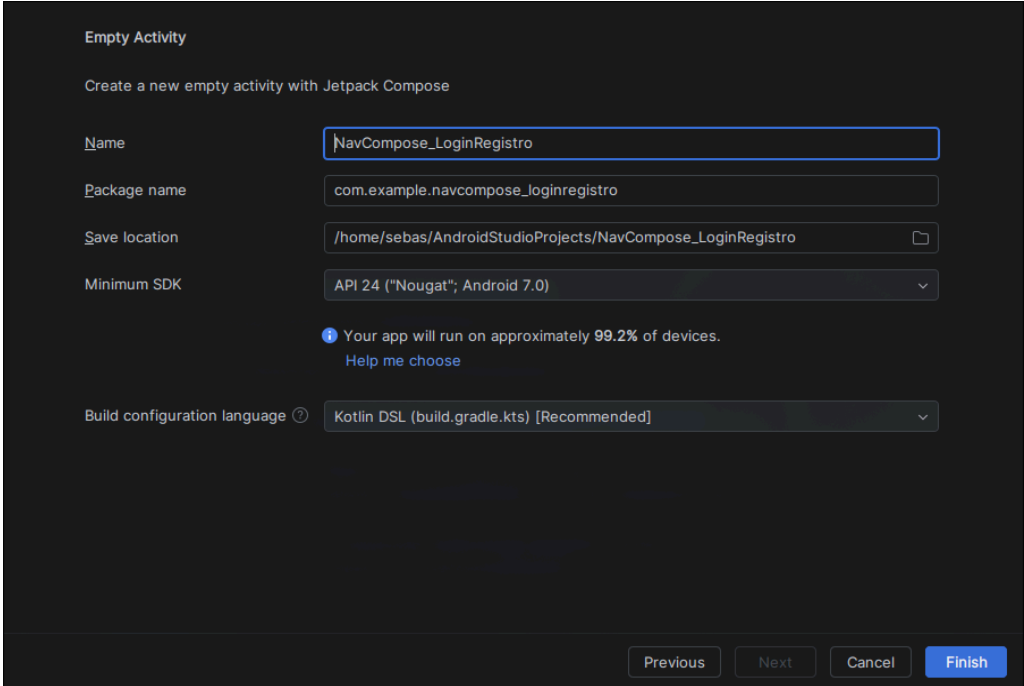


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

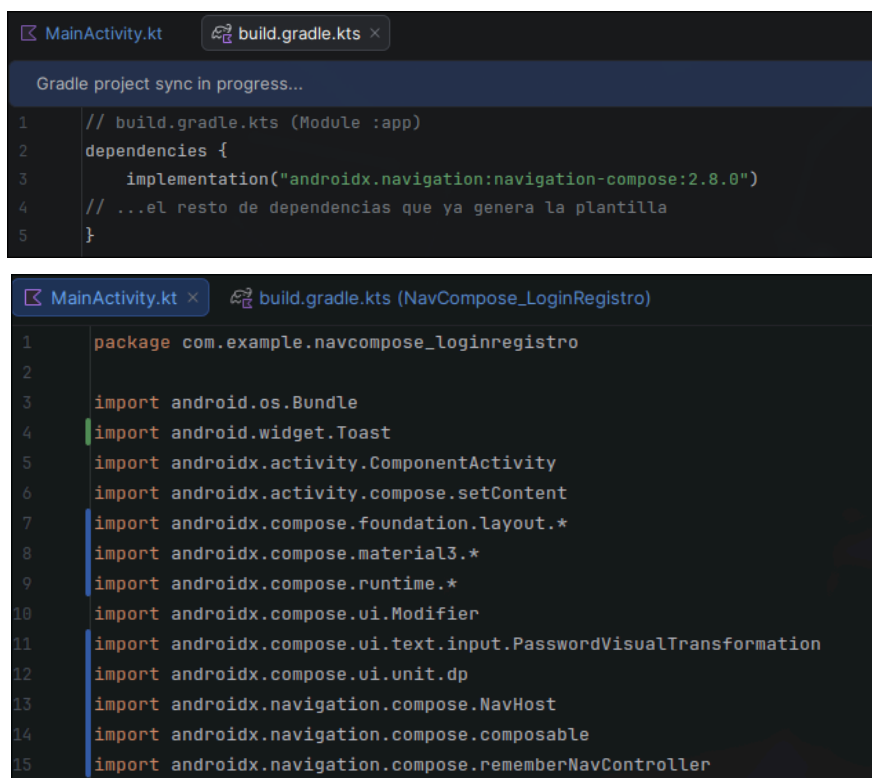
(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCIÓN AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Navegación entre pantallas: inicio de sesión y registro con Jetpack Compose				
NÚMERO DE PRÁCTICA:	3	AÑO LECTIVO:	2026-B	NRO. SEMESTRE:	VIII
FECHA DE PRESENTACIÓN	19/09/2026	HORA DE PRESENTACIÓN	23:59		
INTEGRANTE (s): - Zuñiga Villacorta, Peter Sebastian				NOTA (0-20)	
DOCENTE(s): Roxana Evelyn Limache Calatayud					

RESULTADOS Y PRUEBAS
Github Repo: https://github.com/UNSA-IS-2026-B/idnp_lab/tree/main/lab03 1. EJERCICIOS RESUELTOS Paso 1. Crear un nuevo proyecto con la plantilla "Empty Activity". 

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

Paso 2. Agregar la dependencia de Navigation Compose en build.gradle.kts (módulo app):
implementation("androidx.navigation:navigation-compose:2.8.0") (o la versión estable más reciente), y sincronizar Gradle.



```

1 // build.gradle.kts (Module :app)
2 dependencies {
3     implementation("androidx.navigation:navigation-compose:2.8.0")
4     // ...el resto de dependencias que ya genera la plantilla
5 }

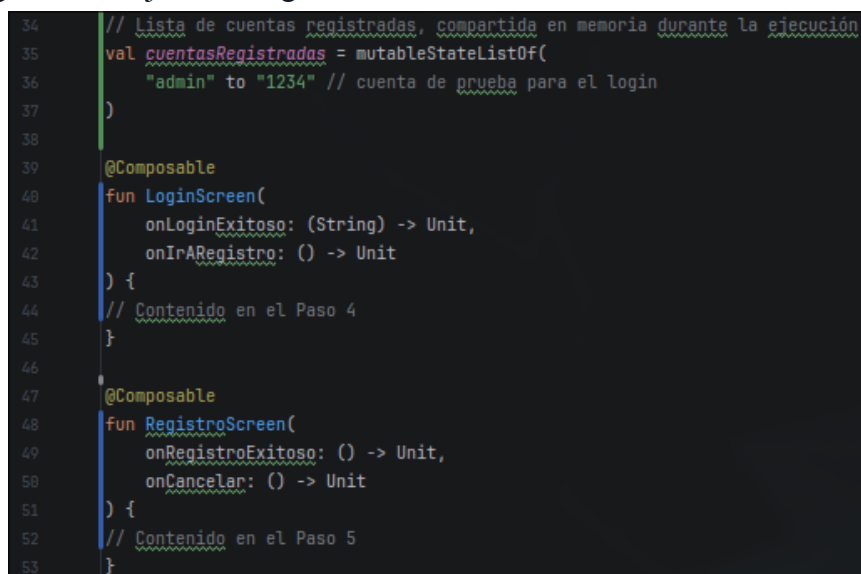
```

```

1 package com.example.navcompose_loginregistro
2
3 import android.os.Bundle
4 import android.widget.Toast
5 import androidx.activity.ComponentActivity
6 import androidx.activity.compose.setContent
7 import androidx.compose.foundation.layout.*
8 import androidx.compose.material3.*
9 import androidx.compose.runtime.*
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.text.input.PasswordVisualTransformation
12 import androidx.compose.ui.unit.dp
13 import androidx.navigation.compose.NavHost
14 import androidx.navigation.compose.composable
15 import androidx.navigation.compose.rememberNavController

```

Paso 3. Crear dos funciones @Composable: LoginScreen(onLoginExitoso, onIrARegistro) y RegistroScreen(onRegistroExitoso, onCancelar). Cada una recibe funciones lambda para comunicar lo ocurrido, en lugar de manejar la navegación internamente.



```

34 // Lista de cuentas registradas, compartida en memoria durante la ejecución
35 val cuentasRegistradas = mutableStateListOf(
36     "admin" to "1234" // cuenta de prueba para el login
37 )
38
39 @Composable
40 fun LoginScreen(
41     onLoginExitoso: (String) -> Unit,
42     onIrARegistro: () -> Unit
43 ) {
44     // Contenido en el Paso 4
45 }
46
47 @Composable
48 fun RegistroScreen(
49     onRegistroExitoso: () -> Unit,
50     onCancelar: () -> Unit
51 ) {
52     // Contenido en el Paso 5
53 }

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

Paso 4. Dentro de LoginScreen, agregar dos TextField (usuario y contraseña, esta última con `visualTransformation = PasswordVisualTransformation()`) y dos botones: "Ingresar" y "Crear cuenta".

```
Kotlin
@Composable
fun LoginScreen(
    onLoginExitoso: (String) -> Unit,
    onIrARegistro: () -> Unit
) {
    var usuario by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    var mensajeError by remember { mutableStateOf("") }
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text("Iniciar sesión", style = MaterialTheme.typography.headlineSmall)
        Spacer(modifier = Modifier.height(16.dp))
        OutlinedTextField(
            value = usuario,
            onChange = { usuario = it },
            label = { Text("Usuario") },
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(8.dp))
        OutlinedTextField(
            value = password,
            onChange = { password = it },
            label = { Text("Contraseña") },
            visualTransformation = PasswordVisualTransformation(),
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(16.dp))
        if (mensajeError.isNotEmpty()) {
            Text(mensajeError, color = MaterialTheme.colorScheme.error)
            Spacer(modifier = Modifier.height(8.dp))
        }
        Button(
            onClick = { onLoginExitoso(usuario) }, // se completa en el Paso 8
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Ingresar")
        }
        Spacer(modifier = Modifier.height(8.dp))
        OutlinedButton(
            onClick = onIrARegistro,
            modifier = Modifier.fillMaxWidth()
        ) {

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

```

        Text("Crear cuenta")
    }
}

```

Paso 5. Dentro de RegistroScreen, agregar los TextField necesarios para los datos del nuevo usuario, y dos botones: "Aceptar" y "Cancelar".

```

Kotlin
@Composable
fun RegistroScreen(
    onRegistroExitoso: () -> Unit,
    onCancelar: () -> Unit
) {
    var usuario by remember { mutableStateOf("") }
    var password by remember { mutableStateOf("") }
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text("Crear cuenta", style = MaterialTheme.typography.headlineSmall)
        Spacer(modifier = Modifier.height(16.dp))
        OutlinedTextField(
            value = usuario,
            onChange = { usuario = it },
            label = { Text("Nuevo usuario") },
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(8.dp))
        OutlinedTextField(
            value = password,
            onChange = { password = it },
            label = { Text("Nueva contraseña") },
            visualTransformation = PasswordVisualTransformation(),
            modifier = Modifier.fillMaxWidth()
        )
        Spacer(modifier = Modifier.height(16.dp))
        Button(
            onClick = {
                cuentasRegistradas.add(usuario to password)
                onRegistroExitoso()
            },
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Aceptar")
        }
    }
}

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```

    }
    Spacer(modifier = Modifier.height(8.dp))
    OutlinedButton(
        onClick = onCancel,
        modifier = Modifier.fillMaxWidth()
    ) {
        Text("Cancelar")
    }
}

```

Paso 6. En MainActivity, dentro de setContent { }, crear el controlador con val navController = rememberNavController() y declarar NavHost(navController, startDestination = "login") registrando ambas pantallas: composable("login") { LoginScreen(...) } y composable("registro") { RegistroScreen(...) }, como se muestra en la Figura 1.

Kotlin

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            val navController = rememberNavController()
            NavHost(navController = navController, startDestination = "login") {
                composable("login") {
                    LoginScreen(
                        onLoginExitoso = { usuario ->
                            Toast.makeText(
                                this@MainActivity,
                                "Bienvenido $usuario",
                                Toast.LENGTH_SHORT
                            ).show()
                        },
                        onIrARegistro = {
                            navController.navigate("registro")
                        }
                    )
                }
                composable("registro") {
                    RegistroScreen(
                        onRegistroExitoso = {
                            navController.popBackStack()
                        },
                        onCancel = {
                            navController.popBackStack()
                        }
                    )
                }
            }
        }
    }
}

```

```

    }
  }
}

```

Paso 7. Conectar la navegación: el botón "Crear cuenta" de LoginScreen debe llamar a `navController.navigate("registro")`; el botón "Cancelar" de RegistroScreen debe llamar a `navController.popBackStack()`.

```

composable( route = "login") {
    LoginScreen(
        onLoginExitoso = { usuario ->
            Toast.makeText(
                context = this@MainActivity,
                text = "Bienvenido $usuario",
                duration = Toast.LENGTH_SHORT
            ).show()
        },
        onIrARegistro = {
            navController.navigate( route = "registro")
        }
    )
}

```

```

composable( route = "registro") {
    RegistroScreen(
        onRegistroExitoso = {
            navController.popBackStack()
        },
        onCancel = {
            navController.popBackStack()
        }
    )
}

```

Paso 8. Implementar una validación básica en LoginScreen: al presionar "Ingresar", comparar los valores ingresados contra una lista de cuentas guardada en memoria (por ejemplo, una `List<Pair<String, String>>`). Si coincide, mostrar "Bienvenido <usuario>"; si no, mostrar un mensaje de error, siguiendo el flujo resumido en la Figura 2.

```

92      Button(
93          onClick = {
94              val coincide = cuentasRegistradas.any {
95                  it.first == usuario && it.second == password
96              }
97              if (coincide) {
98                  mensajeError = ""
99                  onLoginExitoso(usuario)
100              } else {
101                  mensajeError = "Usuario o contraseña incorrectos"
102              }
103          },
104          modifier = Modifier.fillMaxWidth()
105      ) {
106          Text("Ingresar")
107      }

```

Paso 9. Ejecutar el proyecto y verificar el flujo completo: iniciar en LoginScreen, navegar a RegistroScreen con "Crear cuenta", regresar con "Cancelar", e iniciar sesión con credenciales válidas e inválidas.

Iniciar sesión

Usuario

Test

Contraseña

....

Usuario o contraseña incorrectos

Ingresar

Crear cuenta

Crear cuenta

Nuevo usuario

Test

Nueva contraseña

....

Aceptar

Cancelar

Iniciar sesión

Usuario

Test

Contraseña

....

Ingresar

Crear cuenta



Bienvenido Test

2. EJERCICIOS PROPUESTOS

Tomando como referencia el ejercicio resuelto por el docente, completa la aplicación de inicio de sesión y registro agregando persistencia de datos y una pantalla de bienvenida:

1. Agrega una tercera pantalla, HomeScreen(usuario), registrada en el NavHost con la ruta "home", que reciba el nombre del usuario como argumento de navegación (por ejemplo `navController.navigate("home/$usuario")`) y lo muestre en el mensaje "Bienvenido <usuario>".

Se definió la ruta "home/{usuario}" dentro del NavHost.

```
// Ruta 3: Pantalla de Inicio (Home) con argumento en la URL/route
composable(
    route = "home/{usuario}",
    arguments = listOf(
        navArgument( name = "usuario" ) { type = NavType.StringType }
    )
) { backStackEntry ->
    val usuarioArg = backStackEntry.arguments?.getString( key = "usuario" ) ?: ""
    HomeScreen(
        usuario = usuarioArg,
        onCerrarSesion = {
            navController.popBackStack( route = "login", inclusive = false)
        }
    )
}
```

Al autenticar un usuario en LoginScreen, se invoca `navController.navigate("home/$usuario")`.

```
// Ruta 1: Pantalla de Login
composable( route = "login" ) {
    LoginScreen(
        onLoginExitoso = { usuario ->
            // Navega pasando el parámetro en la ruta
            navController.navigate( route = "home/$usuario" )
        },
        onIrARegistro = {
            navController.navigate( route = "registro" )
        }
    )
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

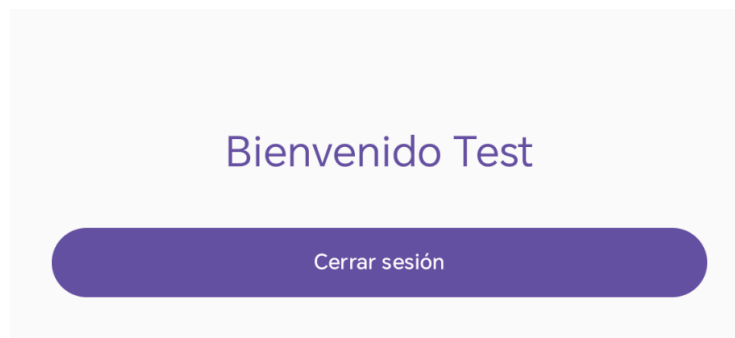
El NavHost extrae la variable desde `backStackEntry.arguments?.getString("usuario")` y se la provee a HomeScreen.

```

) { backStackEntry ->
    val usuarioArg = backStackEntry.arguments?.getString( key = "usuario") ?: ""
    HomeScreen(
        usuario = usuarioArg,
        onCerrarSesion = {
            navController.popBackStack( route = "login", inclusive = false)
        }
    )
}

```

Ejecución pantalla HomeScreen(usuario)



2. En RegistroScreen, al presionar "Aceptar", guarda los datos ingresados (usuario y contraseña) en un archivo de texto plano `cuentas.txt` en el almacenamiento interno de la aplicación (`openFileOutput`, modo `MODE_APPEND`), y regresa a LoginScreen mostrando un mensaje de confirmación.

Se hace uso de `context.openFileOutput("cuentas.txt", Context.MODE_APPEND)` para abrir el archivo en modo anexo.

```

fun guardarCuentaEnArchivo(context: Context, usuario: String, pass: String): Boolean {
    return try {
        val linea = "$usuario:$pass\n"
        context.openFileOutput( name = "cuentas.txt", mode = Context.MODE_APPEND).use { output ->
            output.write( b = linea.toByteArray())
        }
        true
    } catch (e: Exception) {
        e.printStackTrace()
        false
    }
}

```

Se escribe cada credencial con el formato usuario:contraseña\n.

```
val linea = "$usuario:$pass\n"
context.openFileOutput( name = "cuentas.txt", mode = Context.MODE_APPEND).use { output ->
    output.write( b = linea.toByteArray())
}
```

Tras guardar, se muestra un mensaje Toast confirmando el registro exitoso y se retorna a LoginScreen mediante navController.popBackStack().

```
// Ruta 2: Pantalla de Registro
composable( route = "registro") {
    RegistroScreen(
        onRegistroExitoso = {
            navController.popBackStack()
        },
        onCancel = {
            navController.popBackStack()
        }
    )
}
```

Archivo cuentas.txt en almacenamiento interno

com.example.navcompose_loginregistro	drwxrwx--x	2026-09-17 08:27	4 KB
cache	drwxrws--x	2026-09-15 08:31	3.4 KB
code_cache	drwxrws--x	2026-09-19 14:53	3.4 KB
files	drwxrwx--x	2026-09-19 14:53	3.4 KB
cuentas.txt	-rw-rw----	2026-09-19 14:53	10 B
profileInstalled	-rw-----	2026-09-19 14:53	24 B

cuentas.txt	MainActivity.kt
1	Test:Test
2	

- En LoginScreen, al presionar "Ingresar", busca una coincidencia en el archivo cuentas.txt (en lugar de la lista en memoria usada en el ejercicio resuelto). Si existe, navega a HomeScreen; si no, muestra el mensaje "Cuenta no encontrada".

Se utiliza context.openFileInput("cuentas.txt") para leer el archivo línea por línea.

```
context.openFileInput( name = "cuentas.txt").bufferedReader().useLines { lines ->
    lines.any { line ->
        val partes = line.split( ...delimiters = ":")
        partes.size == 2 && partes[0].trim() == usuario.trim() && partes[1].trim() == pass.trim()
    }
}
```

Se verifica si la combinación de usuario y contraseña coincide con algún registro. Si existe coincidencia, navega a HomeScreen; de lo contrario, muestra el mensaje de error "Cuenta no encontrada".

```
// Validación en archivo de texto interno
val coincide = validarCuentaEnArchivo(context, usuario, password)
if (coincide) {
    mensajeError = ""
    onLoginExitoso(usuario)
} else {
    mensajeError = "Cuenta no encontrada"
}
```

4. Agrega validación de campos vacíos en ambos formularios, mostrando un mensaje de error si el usuario intenta enviar el formulario incompleto.

Tanto en LoginScreen como en RegistroScreen, se evalúa mediante `.isBlank()` si los campos están vacíos antes de ejecutar cualquier operación de lectura/escritura.

```
// Validación: Campos vacíos
if (usuario.isBlank() || password.isBlank()) {
    mensajeError = "Por favor, ingrese usuario y contraseña."
}
```

Si falta algún dato, se activa el estado `mensajeError` impidiendo el flujo hasta que el usuario ingrese la información.

Iniciar sesión

Usuario

Contraseña

Por favor, ingrese usuario y contraseña.

Ingresar

Crear cuenta

Crear cuenta

Nuevo usuario

Nueva contraseña

Todos los campos son obligatorios.

Aceptar

Cancelar

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

CUESTIONARIO:

1. ¿Qué ventajas tiene usar Navigation Compose (una sola Activity con varias pantallas) frente al enfoque tradicional de múltiples Activities comunicadas por Intent?

El enfoque de Single Activity Architecture promovido por Navigation Compose ofrece diversas ventajas sobre el modelo tradicional de múltiples Activities.

- Rendimiento y menor sobrecarga de recursos: Crear y destruir un Activity implica la asignación e inicialización pesada de recursos a nivel de sistema operativo (creación de ventanas, llamadas al compilador de recursos, etc.). En cambio, las pantallas en Compose son simples funciones @Composable que se reconfiguran o destruyen en memoria como nodos ligeros de UI.
- Consistencia en componentes globales: Facilita el uso de elementos de interfaz persistentes (como una BottomNavigationBar, un NavigationDrawer o una TopAppBar) sin necesidad de duplicar vistas o gestionar su estado independientemente entre diferentes actividades.
- Animaciones y transiciones fluidas: Permite realizar transiciones de pantalla compuestas y animaciones de entrada/salida de forma nativa e integrada en la arquitectura declarativa de Jetpack Compose.
- Gestión de estado simplificada: Elimina la complejidad de manejar múltiples ciclos de vida independientes y la necesidad de serializar objetos grandes en cada transición a través de Intent o Bundle, integrándose directamente con la arquitectura de Unidirectional Data Flow (UDF).

2. Además de los argumentos de navegación, ¿qué otro mecanismo podrías usar para compartir datos entre pantallas en una app Compose (por ejemplo, un ViewModel compartido)? Explica brevemente cómo funcionaría.

Aparte de los argumentos en las rutas, el mecanismo recomendado por la guía de arquitectura oficial de Android para compartir información entre pantallas es un ViewModel compartido (Shared ViewModel) acoplado al grafo de navegación o al ciclo de vida de la Activity principal.

¿Cómo funciona?

Definición del ViewModel: Se crea una clase derivada de ViewModel que contiene los datos a compartir mediante estados reactivos (por ejemplo, StateFlow, SharedFlow o mutableStateOf).

Kotlin

```
class SharedUserViewModel : ViewModel() {
    var usuarioActual by mutableStateOf<Usuario?>(null)
    private set

    fun seleccionarUsuario(usuario: Usuario) {
        usuarioActual = usuario
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

```
}
}
```

Alcance (Scope) Compartido: En lugar de instanciar un ViewModel distinto por cada pantalla composable, se obtiene la instancia del ViewModel delimitada al contenedor superior o al NavBackStackEntry del grafo correspondiente:

Kotlin

```
// Dentro de MainActivity o del NavHost
val sharedViewModel: SharedUserViewModel = viewModel()

NavHost(navController = navController, startDestination = "login") {
    composable("login") {
        LoginScreen(onLogin = { user ->
            sharedViewModel.seleccionarUsuario(user)
            navController.navigate("home")
        })
    }
    composable("home") {
        // Accede al mismo ViewModel y a su estado actualizado
        val usuario = sharedViewModel.usuarioActual
        HomeScreen(usuario = usuario?.nombre ?: "Invitado")
    }
}
```

Ventajas clave del Shared ViewModel:

- Evita pasar objetos complejos por URL/Ruta: Las rutas de navegación solo deben transportar identificadores mínimos (ejemplo: un id numérico o String), mientras que los datos complejos o estructuras pesadas se mantienen centralizados en la capa de datos expuesta por el ViewModel.
- Reactividad: Cuando el ViewModel actualiza su estado en una pantalla, cualquier otra pantalla que esté observando dicho estado se recompondrá de forma automática.

CONCLUSIONES

1. El uso de Navigation Compose y la arquitectura de una sola actividad demostró ser una solución eficiente para gestionar el flujo de la aplicación, permitiendo administrar la transición entre pantallas como componentes composables ligeros en un contenedor centralizado `NavHost` y optimizando el uso de recursos frente al enfoque tradicional.
2. La implementación de las pantallas de inicio de sesión y registro se realizó de forma independiente y modular mediante la técnica de **State Hoisting**, delegando las acciones de navegación a través de funciones lambda y manteniendo el código de la interfaz totalmente desacoplado de la lógica de rutas.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

3. El paso de datos entre pantallas se aplicó correctamente mediante la definición de rutas parametrizadas en el `NavController`, lo que permitió enviar la información del usuario autenticado hacia la pantalla de bienvenida de manera limpia y consumirla dinámicamente según el contexto de navegación.

REFERENCIAS Y BIBLIOGRAFÍA

- [1] [1] Android Developers, "Navigation con Compose". [En línea]. Disponible: <https://developer.android.com/develop/ui/compose/navigation>
- [2] Android Developers, "Aprende Android". [En línea]. Disponible: <https://developer.android.com/teach>
- [3] Android Developers, "Guías". [En línea]. Disponible: <https://developer.android.com/guide>
- [4] "Navegación en Jetpack Compose con Navigation Compose", video. [En línea]. Disponible: <https://www.youtube.com/watch?v=nwde1ct06TI>